which is the somewhat odd construct that division by zero gets you in Ruby. You also need to make sure projected_days_remaining doesn't raise an exception if there are no tasks.

You can use the same predicate in the code to make the on_schedule? assertion pass like this:

basics/08/app/models/project.rb
```
def on_schedule?
  return false if projected_days_remaining.nan?
  (Time.zone.today + projected_days_remaining) <= due_date
end
```

And the tests pass, which brings you to the refactoring phase. The first thing to notice is that you have a duplicated piece of data: the 21-day window for determining whether a task counts toward velocity. This data point is referenced in both Project#current_rate and Task#part_of_velocity?. They are pretty clearly the same bit of data—if I changed the time period to two weeks, I'd have to change it in both places.

That said, it's not clear what to do with this information. To me, the velocity length feels most like a static constant value owned by the Project class, since velocity applied to a single task makes no sense. In code you can do this by implementing the velocity_length_in_days as a class method with a constant return value:

basics/08/app/models/project.rb
```
def self.velocity_length_in_days
  21
end
```

I'm using a method rather than a constant because this seems likely to become dynamic at some point in the future. Using a method preserves the API at no additional complexity cost.

The one usage of velocity length in the Project class changes to this:

basics/08/app/models/project.rb
```
def current_rate
  completed_velocity * 1.0 / Project.velocity_length_in_days
end
```

And the one usage of velocity length in Task is now as follows:

basics/08/app/models/task.rb
```
def part_of_velocity?
  return false unless complete?
  completed_at > Project.velocity_length_in_days.days.ago
end
```